Building together

The DiracX Transformation System

DiracX hackathon · 12th DIRAC(X) Users' Workshop · FZU Prague



From a game to working code

TUE 16:10

Ice-breaker game

40 min

Run a workgraph by hand, with sticky notes.

TUE 16:50

1-2-4-All

1 h

What must exist, and how do we split the work?

WED 14:00

Mob programming

1 h 50

We build the database and the state machines together, with Claude.

WED 16:10

Planning poker

45 min

We size the work items and form pairs.

WED 17:00 → FRI

We build

about 6 h

Pairs and small pull requests. Demo on Friday.

> Code is becoming cheap.
> What we need is a **common understanding** of what to build, and of how we **review** it.

1 · Ice-breaker game

Run a Monte Carlo workgraph by hand

The game: 8 seeds become 1 merged file

Seed stack
8 red notes, S1 … S8

→ **SIMULATION**
1 seed per parcel

→ **RECONSTRUCTION**
2 green per parcel

→ **MERGE**
4 blue per parcel

→ **Finished**
1 yellow note in Merge's Outputs: shout **“Completed!”**

THREE PLAYERS PER TABLE, ONE BOARD AT A TIME: SIMULATION, THEN RECONSTRUCTION, THEN MERGE

Feeder → Pool (at most 2 notes at a time) → **Packer** → **Compute backend** → ✓ Done (new notes → Outputs)

✗ the note goes back to the Pool, a **new** parcel takes it

dark a player dashed a place on the table **Parcel**: a quarter sheet with its input notes stuck on it, one job

Round 1

ROUND

1

7 min

- Read your sheet. Ask your team, not us.
- Start when we say **go**.
- One board at a time: 8 seeds → 8 green → 4 blue → **1 yellow**.
- Shout **“Completed!”** when the yellow note is in Merge's Outputs.

Round 2

ROUND

2

7 min

- Reset the table: notes back in the stack, new parcels.
- **Same game, same goal.**
- Shout **“Completed!”** when the yellow note is in Merge's Outputs.

Round 3

ROUND

3

7 min

- **Change roles:** Feeder → Packer → Compute backend → Feeder. Take the sheet of your new role.
- The Packer receives a **new version of the Reconstruction packer plugin**.
- Same goal. Shout **“Completed!”**

Debrief

What happened

- Where did you **wait**? Who was the bottleneck?
- When a job failed, what was **retried**: the parcel or the seed?
- How did you know the work was **finished**?
- Who knew how many notes were where? Now imagine **100 million** notes.

YOU PLAYED	WE WILL BUILD
Feeder	**Feeder, edge feeder**
Packer	**Packer**
Packer → backend	**Dispatcher**
Backend: the job	**Dummy backend**
Backend: ✓ and ✗	**Completion**
Nobody kept the books	**Database, counters**
The rules on the boards	**CWL submission**

> In the game, you moved the notes yourselves. In DiracX, a feeder or a packer only **says** what should happen. The core then updates the database in one step, so a note is never lost or counted twice.

2 · 1-2-4-All

What must exist, and how do we split it?

1-2-4-All

> What must exist for the whole Transformation System to work? What do we build first? What is unclear?

Already known: 2-week sprints · 4 to 5 pairs · the dummy executor exists and must be adapted to parcels · `simulation.cwl` in the simulator

1

`5 min · alone`

One piece of work per note 🟨, one question per note 🟥. In silence.

2

`10 min · in pairs`

Merge your notes, remove duplicates, sharpen the words.

4

`15 min · in fours`

Group into **work items**: one behaviour, one pair, one sprint. Say when it is **done**.

All

`25 min · together`

All items on the wall, in order, to the full system. Add only what is new. Pick the first sprint.

Outcome: the whole system in order; the first sprint as items, sized on Wednesday.

Example: *a failed parcel sends its inputs to Failed* · done: a test sees them Failed, ErrorCount 1 · needs: transitions

3 · Mob programming

The shared core, built together

Mob programming: the shared core, with Claude

WHAT WE BUILD

- database tables · DX-ADR-004
- state machines as data · DX-ADR-005
- transitions module: the only code that changes a status
- one transition + its test

1 · THE ROOM
Decides the next prompt
architects check it against the ADRs

→

2 · ONE PERSON
Types it into Claude
on the projector

→

3 · GIT
One prompt, one commit
pushed to a draft pull request

→

4 · EVERYONE
Reviews the diff live
and comments on GitHub

the comments become the next prompt

> Every pair will build on this code, so we learn it together.

4 · Planning poker

How big is each item?

Planning poker

1	2	3	5	8	13	?
Trivial. We know exactly what to change.	Small and clear.	Medium. Some thinking, few unknowns.	Large. Several parts, some unknowns.	Very large. Try to split it.	Too big. Split it before we start.	I do not understand the item.
 1 finger	 2 fingers	 3 fingers	 one hand	 5 + 3	 arms crossed	 fist

- Points measure **size**: effort, complexity, uncertainty. Not hours.
- “3, 2, 1, **show**”: everyone shows their fingers at the same time.
- The **highest** and the **lowest** explain. Then one more vote.
- Our usual pace: about **6 points per person per sprint**.

Now, let’s build it

- Pick your first item
- Small pull requests to `main`
- Review another pair's pull request before your next item
- Demo on Friday

Backup

Game material: set-up, table, role sheets

Game set-up and timing

Tables of 3 players

- 3 players per table: Feeder, Packer, Compute backend. About 10 people: 3 tables, one observer.
- Per table: red 🟥, green 🟩, blue 🟦 and yellow 🟨 notes; 8 red notes written in advance, `S1` … `S8`
- Per table: pens, scrap paper cut in quarters (the parcels), 3 boards, 3 role sheets, the two cards on the last page
- Pink and yellow pads for the 1-2-4-All

TIME	STEP
3 min	**Explain the game**
5 min	**Read sheets, set up the table**
7 min	**Round 1, happy path**
2 + 7 min	**Reset, round 2, a surprise failure**
2 + 7 min	**Reset, round 3, roles rotate and new packer rule**
7 min	**Debrief**

The table

SIMULATION board

1 red per parcel

Pool	Done	Outputs

RECONSTRUCTION board

2 green per parcel

Pool	Done	Outputs

MERGE board

4 blue per parcel

Pool	Done	Outputs

Seed stack
S1 … S8, written in advance

Backend seat
pads, pens; one parcel at a time

Bin
every used parcel, done or failed. Never reused.

A parcel is a quarter sheet with the input notes stuck on it (`P5 · RECO`). After the job, the Compute backend puts ✓ inputs in Done, new notes in Outputs, ✗ inputs back in their Pool, and the empty parcel in the Bin.

Only the seeds have names, written in advance. The other notes stay blank: green is what Simulation makes, blue Reconstruction, yellow Merge.

Feeder

You play the **feeder** plugin of all three transformations, one board at a time. On Reconstruction and Merge you are an **edge feeder**: your inputs are the outputs of the board before.

You hold: the seed stack S1 … S8. Never more than **2 notes** in your hands.

1. **Simulation:** move seeds 🟥 from the stack into its Pool, **at most 2 at a time**. Stack empty? Say **“Simulation feed is over.”**
2. **Reconstruction**, once Simulation's Done holds all 8 seeds: move the green notes 🟩 from Simulation's Outputs into its Pool, at most 2 at a time. Outputs empty? Say **“Reconstruction feed is over.”**
3. **Merge**, once Reconstruction's Done holds all 8 green notes: the same with the blue notes 🟦. Then say **“Merge feed is over.”**

Packer

You play the **packer** plugin of all three transformations, one board at a time: you decide which notes go together in one job.

You hold: paper quarters, a pen. A rule may change in round 3.

1. Take notes from the Pool of the current board: **Simulation** 1 seed per parcel, **Reconstruction** 2 green, **Merge** 4 blue.
2. Write `P1 · SIM` (then P2, P3…) on a paper quarter and stick the notes on it.
3. Hand the parcel to the **Compute backend**. It already holds one? Wait.
4. Not enough notes? Wait. Once the Feeder says the feed is over, pack what is left.

Compute backend

You play where the jobs run, for all three transformations, and you record what each job did on the boards. One parcel at a time.

You hold: the  pads, a pen. If you get a card, read it in secret.

1. **Run:** take blank notes of the next colour, nothing to write: one green 🟩 per **SIM** parcel, one blue 🟦 per **RECO** parcel, one yellow 🟨 per **MERGE** parcel.
2. **Record:** ✓ inputs go to the board's **Done**, the new notes to its **Outputs**. A ✗ input goes back to its **Pool**.
3. Put the empty parcel in the **Bin**. A parcel is never reused.
4. When the yellow note is in Merge's Outputs and every Pool is empty, shout **“Completed!”**

Simulation board

1 red per parcel · print on A4 landscape

Pool

waiting to be packed

In: Feeder (seeds), Compute backend (✗ inputs)
Out: Packer

Done

processed by exactly one parcel

In: Compute backend only (✓ inputs)
Out: nobody

Outputs

green notes, for the next board

In: Compute backend only (new notes)
Out: the Feeder takes them to Reconstruction

Reconstruction board

2 green per parcel · print on A4 landscape

Pool

waiting to be packed

In: Feeder (green notes), Compute backend (✗ inputs)
Out: Packer

Done

processed by exactly one parcel

In: Compute backend only (✓ inputs)
Out: nobody

Outputs

blue notes, for the next board

In: Compute backend only (new notes)
Out: the Feeder takes them to Merge

Merge board

4 blue per parcel · print on A4 landscape

Pool

waiting to be packed

In: Feeder (blue notes), Compute backend (✗ inputs)
Out: Packer

Done

processed by exactly one parcel

In: Compute backend only (✓ inputs)
Out: nobody

Outputs

the yellow note: the finished product

In: Compute backend only (new notes)
The yellow note stays here: the game is over

Cards

COMPUTE BACKEND · ROUND 2 · SECRET

A job fails

Parcel `P3 · SIM` fails: make no green note and put its seed back in the Simulation Pool.

When that seed comes back on a new parcel, it works.

PACKER · ROUND 3

New Reconstruction packer

3 green  per parcel, instead of 2.

Once the Feeder says the feed is over, pack what is left.